

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_e1b943dd-dac\agent-ade90b95dc2bdd643.jsonl`  
- SHA-256: `7e5a02599a25f6ad4473b2640f7fcc51ff94156952ccf756189064978a971d5b`  
- Source modified: `2026-07-06T19:19:22+00:00`  
- Imported at: `2026-07-08T16:00:12+00:00`  
- project: `wf_e1b943dd-dac`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T19:18:19.573Z)

Reviewing GitHub PR #40 of the repo at C:/Users/avidu/Projects/Telegram ? a DRAFT, DOCS-ONLY PR (branch docs/track-f-policy-pipeline).
It adds ONE file: docs/adr/0012-policy-pipeline.md (ADR 0012, status Proposed) ? the Track F design doc for a four-tier "policy pipeline" (Tier 1 metadata gates / Tier 2 topic preferences / Tier 3 content safety / Tier 4 deep inspection), a PolicyStage protocol + StageResult explanation trace, and 7 phased follow-up issues F1-F7.
No code changes. Read the ADR:
git -C C:/Users/avidu/Projects/Telegram show docs/track-f-policy-pipeline:docs/adr/0012-policy-pipeline.md

This is a DESIGN review, not a code review ? there is no gate to run. Judge the ADR as a design contract:
soundness, completeness, internal consistency, accuracy of its claims, and consistency with the codebase and the other ADRs (docs/adr/0001..0011 exist on main; read any you need via 'git -C C:/Users/avidu/Projects/Telegram show main:docs/adr/<file>').
Relevant current code (on main): src/tg_video_filter/models.py (FilterConfig, ContentItem, Source, Policy),
src/tg_video_filter/db.py (load_filter_config/save_filter_config shims, sources.watched gate should_ingest_source, delivery_attempts/record_delivery), src/tg_video_filter/telethon_client.py (the live handler: watched gate -> dedup -> FilterEngine.evaluate -> insert -> deliver). Track E (sources.watched opt-in gate) and Track C (delivery_attempts) just merged.

Ground rules:

- READ actual files, cite ADR section / file:line. You MAY run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe to check claims (e.g. does FilterConfig ignore/drop unknown JSON keys? does ContentItem carry caption text? is content_item.text populated?).
- This is a DRAFT asking for design feedback ? frame findings as design gaps/risks/inaccuracies, not merge-blockers.
 - Severity: high (a claim that is false, or a design gap that would break an F-phase / cause data loss), medium (unspecified interaction, inconsistency, missing dependency), low (wording, invalid example, minor).
- Report only issues in THIS ADR. No praise. Be concrete: what's wrong, why it matters for implementation.

ADVERSARIALLY VERIFY this design finding ? try to REFUTE it. Read the cited ADR section + any referenced ADR/code,
run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe checks if useful. CONFIRMED if it genuinely holds against THIS ADR (a real gap/inaccuracy/inconsistency).
REFUTED if the ADR actually addresses it / the claim is wrong / it's out of scope for a design doc. PLAUSIBLE if unsettled. Corrected severity (remember: draft design doc, so calibrate ? false claim/data-loss = high, gap = medium).
FINDING (design): Tier 3 'Duplicate suppression' stage overlaps/contradicts the already-shipped dedup that runs BEFORE the pipeline
severity medium | section Decision ? Security stages (Tier 3) item 3, Duplicate suppression
summary: The ADR places a dedup stage inside Tier 3, but dedup already runs pre-pipeline (the ADR itself says the pipeline runs 'after dedup'), and content_items.dedup_key enforces per-account dedup at the DB level ? so a Tier-3 dedup stage is either redundant or defines a new cross-policy scope the schema doesn't support.
why_it_matters: It claims to add 'cross-policy duplicate detection within an account's delivery streams,' but delivery-stream dedup is a delivery_attempts / destination concern (Track C), not a policy-gate concern, and the pipeline has no visibility into other policies' delivery streams via PolicyContext (which sees one policy). This stage's data source and scope are undefined and it partially re-litigates ADR 0011's dedup decision. Risk: implementers build a second, inconsistent dedup.
evidence: ADR Context: 'pipeline runs after dedup and before delivery.' ADR 0011 Option 1A: UNIQUE(account_id, dedup_key) already enforces per-account dedup. telethon_client.py runs three-tier dedup before FilterEngine.evaluate. PolicyContext exposes only the single owning policy.
rec: Either drop Tier-3 duplicate suppression (covered by pre-pipeline dedup_key) or specify the exact new scope (cross-destination within an account), where it reads that state from (delivery_attempts?), and how the single-policy PolicyContext gains cross-policy visibility. As written it's an unspecified overlap.

2. assistant (2026-07-06T19:18:22.118Z)

I'll verify this finding by reading the ADR and the relevant code/ADRs.

3. user (2026-07-06T19:18:24.169Z)

ADR 0012: Policy pipeline ? explainable, tiered, user-customizable content gates

- **Status**: Proposed
- **Date**: 2026-07-06
- **Decision owner**: project owner
- **Tracking issue**: #31 (Track F of the provider-neutral umbrella, #25)
- **Depends on**: ADR 0011 (Track B ? domain schema, `policies` table)

Context

The system today evaluates every matched video against a single `FilterConfig` (per account, via `policies.config\_json`). The filter is **flat**: all rules run unconditionally, the output is a boolean (`matched`), and there is no way to express topic preferences, security policy, or why a specific video passed or failed.

Track F defines a **policy pipeline** ? an ordered sequence of stages that run before delivery ? so the system can express richer decisions, explain them, and defer expensive work until after cheap rejections.

What we have today (post-Track-E)

Concept	Current state
--- ---	
`FilterConfig` / `Policy`	One JSON blob per account. Flat duration/resolution/size/exclude rules. AND semantics.
`FilterEngine`	Evaluates all rules; returns `FilterResult(passed, reasons)`.
`policies` table	One row per account (is_default=1). `config_json` is the `FilterConfig`

```
blob. |
| Security / topic / deep-inspect | Nothing. Codec/keyword/MIME filters and ffprobe were
deferred to v2 in the original plan. |
```

What we want

A **policy** should be a pipeline of ordered **stages**, each with a well-defined cost, inputs, and explainable output. The pipeline runs **after** dedup and **before** delivery. Users should be able to express topic preferences ("I like AI and philosophy videos"), security rules ("reject links from sketchy domains"), and view **why** a specific item was accepted or rejected.

Decision

Stage model ? four ordered tiers

The pipeline is split into four tiers, ordered by cost (cheapest first). Later stages only run if all earlier stages pass:

```
...
????????????????????????????????????????????????????????????
?          Policy Pipeline                                     ?
?                                                                 ?
? Tier 1 ? Metadata gates  (always runs, zero I/O)           ?
?   duration, resolution, size, codec, mime, GIF/round       ?
?   ? extends today's FilterEngine                           ?
?                                                                 ?
? Tier 2 ? Topic preferences (opt-in, metadata-only)         ?
?   source allowlist, keyword match, channel topic tags      ?
?   ? user-expressed interests filter the stream             ?
?                                                                 ?
? Tier 3 ? Content safety  (opt-in, metadata + light I/O)    ?
?   URL/domain reputation, file metadata sanity,             ?
?   caption text scanning                                    ?
?   ? cheap checks; no download                              ?
?                                                                 ?
? Tier 4 ? Deep inspection (opt-in, heavy I/O)               ?
?   ffprobe, perceptual hash, downloaded content scan       ?
?   ? gated behind explicit user opt-in + budget             ?
????????????????????????????????????????????????????????????
...
```

Stage interface ? `PolicyStage`

Each stage is a pure function (or async function for I/O stages) that takes a `PolicyContext` and returns a `StageResult`:

```
```python
@dataclass
class PolicyContext:
 """Immutable snapshot of what the stage can inspect."""
 content_item: ContentItem # from ADR 0011
 source: Source # the source it came from
 policy: Policy # the owning policy config

@dataclass
class StageResult:
 passed: bool
 reason: str # human-readable explanation
 stage_name: str # e.g. "duration-gate", "url-reputation"
 metadata: dict[str, object] = field(default_factory=dict) # for explainability UI

class PolicyStage(Protocol):
```

```

 """A single gate in the pipeline. Stateless; may be async for I/O stages."""
 async def evaluate(self, ctx: PolicyContext) -> StageResult: ...
...

```

The pipeline evaluates stages in order and **short-circuits** on the first failure (don't waste I/O on an already-rejected item). The accumulated `StageResult`` list is the **explanation trace** ? every decision is auditable.

## Topic preferences ? Tier 2

Topic preferences are stored as a JSON array in `policies.config_json`` under a `topics`` key:

```

```json
{
  "min_duration_s": 5,
  "topics": {
    "include": ["AI", "Philosophy", "Soccer"],
    "exclude": ["Politics", "Crypto"],
    "mode": "any" // "any" = match any include topic; "all" = match all
  }
}
```

```

Topic matching is **metadata-only** in v1: source title substring match, message caption keyword match, and (future) channel topic tags from Telegram's channel metadata. No ML classification in the design ? that's a separate issue.

## Security stages ? Tier 3

Security stages are **opt-in** and configured per-policy. The design defines three initial stages (implementation deferred to separate issues):

1. **Source allowlist**: reject items from sources not in the `allowlist`` (an explicit list of trusted `provider_source_ids`).
2. **URL/domain reputation**: check links in captions against a configurable blocklist. Metadata-only ? no page fetch.
3. **Duplicate suppression**: enhanced dedup that compares by `content_fingerprint`` across accounts (already handled by `content_items.dedup_key``; this stage adds cross-policy duplicate detection within an account's delivery streams).

Each security stage records its decision in `StageResult.metadata`` for explainability (e.g. `{"blocked_domain": "sketchy-site.com"}``).

## Deep inspection ? Tier 4

Deep inspection (ffprobe, perceptual hash, downloaded content scan) is **gated** behind explicit opt-in because it violates ADR 0003's no-download principle. The opt-in is per-policy (`deep_inspect: true`` in `config_json``) and the stage has a **budget** (max bytes per item, max concurrent downloads) enforced by the policy engine.

This tier is explicitly deferred to a follow-up issue ? the design only **reserves the slot** in the pipeline model so it doesn't require a schema migration when implemented.

## Policy evaluation flow (post-dedup, pre-delivery)

```

```python
async def evaluate_pipeline(
    ctx: PolicyContext,

```

```

    stages: list[PolicyStage],
) -> tuple[bool, list[StageResult]]:
    results: list[StageResult] = []
    for stage in stages:
        result = await stage.evaluate(ctx)
        results.append(result)
        if not result.passed:
            return False, results # short-circuit
    return True, results
...

```

The caller (telethon_client or future Bot API handler) records the results via `delivery\_attempts.last\_error` (for failed items) or a new `policy\_decisions` table (for explainability ? deferred to implementation).

Schema ? minimal changes

****No schema migration**** is required for this design. The `policies` table already holds a `config\_json` JSON blob. The pipeline is a code concept:

- `policies.config\_json` gains optional `topics`, `allowlist`, `security\_checks`, `deep\_inspect` keys ? all backwards-compatible with the existing `FilterConfig` Pydantic model (extra fields are ignored).
- Future `policy\_decisions` table (deferred) would store the explanation trace for debugging: `(content\_item\_id, policy\_id, stage\_name, passed, reason, metadata\_json, evaluated\_at)`.

Implementation phasing (future issues)

Phase	Scope	Dependencies
F1	Policy engine prototype	`PolicyStage` protocol, `evaluate_pipeline()`, wire into live-message handler
F2	Topic preferences	`topics` config parsing, source title/caption keyword matching
F3	Source allowlist	`allowlist` config, skip non-allowlisted sources
F4	URL/domain reputation	Domain blocklist, caption link extraction
F5	Deep inspection	ffprobe/perceptual hash, download budget enforcement
F6	Policy decisions table	Audit trail storage, explainability UI
F7	Explainability endpoint	`/why <content_item_id>` command showing the explanation trace

Each phase is a separate, mergeable PR ? none are bundled here. This ADR only defines the ****design contract**** the phases code against.

Consequences

****Positive****

- Unifies topic, security, and quality gates into a single explainable framework ? no ad-hoc if-chains scattered across handlers.
- Cheap gates always run first; expensive enrichment is only reached for items that pass the cheap gates.
- `StageResult` list provides an audit trail (explainability) without additional DB lookups at decision time.
- Backwards-compatible: `policies.config\_json` already exists; adding optional keys requires no migration.
- Each implementation phase is independently shippable behind feature flags.

****Negative****

- Four tiers add conceptual complexity vs. today's flat filter.
- `PolicyStage` protocol requires all stages to conform to the same interface, which may be awkward for deeply different checks

- (metadata-only vs. I/O-heavy). Mitigated by the `metadata` dict catch-all.
- Topic matching is string-based substring matching in Phase 1 ? not semantic. A future ML stage would replace it, but it's good enough for a v1.
- Deep inspection violates ADR 0003 and must be explicitly opt-in with a budget to avoid runaway costs.

Alternatives considered

- ****Do nothing; keep flat FilterConfig.**** Rejected ? the umbrella exists to dissolve the conflation of quality, topic, and security into a single boolean. A pipeline is the natural generalization.
- ****Inline each stage in the handler**** (no protocol). Rejected ? makes it impossible to test stages independently, reorder them, or add new ones without touching the hot path.
- ****Use a rules engine / DSL.**** Rejected ? over-engineered for <100 users. A Python list of callables is simpler, testable, and introspectable.
- ****Bundle all phases into a single PR.**** Rejected ? scope discipline. Each phase is independently shippable; this ADR defines the shared interface they code against.

Open questions for the decision owner

1. ****`StageResult.metadata` shape**** ? dict or typed per-stage? Recommend: dict for Phase 1 (flexible), typed in Phase 2 (once the shape stabilizes).
2. ****Policy decisions table**** ? store every decision, or only failures? Recommend: every decision (explainability), with a TTL-based cleanup for old traces.
3. ****Topic matching**** ? substring only, or regex/glob? Recommend: substring (case-insensitive) for v1; natural-language matching in a follow-up.
4. ****Deep inspection budget**** ? per-item byte limit or per-account bandwidth cap? Recommend: per-item limit (10 MB default) with a per-account monthly cap enforced at the app level.

References

- ADR 0003 ? metadata-first probing (the no-download principle that Tier 4 must opt in to violate).
- ADR 0009 ? selective channel scanning (the `topics.allowlist` concept generalizes `sources.watched`).
- ADR 0011 ? provider-neutral domain schema (the `policies` table this design extends, `ContentItem` as the `PolicyContext` input).
- Issue #31 ? Track F (this ADR's tracking issue).
- Issue #25 ? umbrella; this is the final track.

4. assistant (2026-07-06T19:18:27.587Z)

Now let me read the referenced ADR 0011 and the relevant code to verify the claims about dedup scope and PolicyContext visibility.

5. user (2026-07-06T19:18:29.845Z)

ADR 0011: Provider-neutral domain schema (Track B)

- ****Status****: Accepted (Question 1 ? Option 1A; Question 2 ? Option 2A; PR #34)
- ****Date****: 2026-07-06
- ****Decision owner****: project owner
- ****Tracking issue****: #27 (Track B of the provider-neutral umbrella, #25)

- **Depends on**: ADR 0010 (Track A ? `accounts` + `telegram\_sessions`, merged in #33)
- **Blocks**: #28 (Track C ? delivery/routing), #29 (Track D ? commands), and indirectly #30 (Track E)

Context

ADR 0010 split `users` into `accounts` (product identity) + `telegram\_sessions` (MTPROTO credential). That unblocked the *identity* dimension. This ADR tackles the **content dimension**: how to represent sources, content items, policies, destinations, routes, and delivery attempts in a provider-neutral way, so that the system can ingest from Telegram today and from WhatsApp (or any future provider) later without another schema rewrite.

What we have today (post-Track-A)

Table	Meaning	Provider-coupling
`accounts`	Product identity (ADR 0010)	provider-neutral (`provider` column)
`telegram_sessions`	Optional MTPROTO session (ADR 0010)	Telegram-specific by design
`channels`	A Telegram channel/chat an account is a member of	Telegram-specific (`channel_id` is a Telegram id)
`videos`	A seen Telegram video message + metadata	Telegram-specific (`message_id`, `document_id` are Telegram ids)
`filter_config`	Per-account JSON blob of `FilterConfig`	provider-neutral in shape, but scoped per-account not per-source

The dedup logic (ADR 0006) is **3-tier and Telegram-specific**:

- Tier 0: `(user\_id, channel\_id, message\_id)` ? same Telegram message.
- Tier 1: `(user\_id, document\_id)` ? same Telegram file forwarded across channels.
- Tier 2: `(user\_id, size\_bytes, duration\_s, height, width)` ? re-upload fingerprint.

What we need

The umbrella (#25) introduces six concepts: `Source`, `ContentItem`, `Policy`, `Destination`, `Route` (`routing\_rules`), and `DeliveryAttempt`. This ADR defines the schema for each, the **two open design questions** that gate the schema, and the migration path from the current tables.

The two decisions I need from you

Question 1: `ContentItem.dedup\_key` ? how does provider-neutral dedup relate to ADR 0006?

The current 3-tier dedup is Telegram-specific (`document\_id` is a Telegram concept). A provider-neutral `ContentItem` needs a dedup story that works across providers. Three options:

Option 1A (recommended): tiered lookup that wraps ADR 0006's tiers

Keep ADR 0006's three tiers but generalize them behind a single `dedup\_key` column on `content\_items`:

```
...
content_items.dedup_key TEXT NOT NULL
...
```

The *value* of `dedup\_key` is computed by an explicit decision tree at ingestion time, provider by provider. The tree is ordered strongest-key first and each branch is **guarded** so partial metadata can never collapse to a degenerate key (e.g. `tg:fp:0:0:0:0`):

- **Telegram**:
 1. If `document\_id` is present ? `dedup\_key = "tg:doc:{document\_id}"`.
 2. Else if **all four** fingerprint fields (`size\_bytes`, `duration\_s`,

`height`, `width`) are present ? `dedup\_key = "tg:fp:{size}:{dur}:{h}:{w}"`.

3. Else ? `dedup\_key = "tg:msg:{source\_id}:{provider\_message\_id}"`
 (Tier 0; the source/channel id is required because Telegram
 `message\_id` is only unique within a channel, not across sources).

- **Future WhatsApp:** `dedup\_key = "wa:msg:{message\_id}"` (WhatsApp message
 ids are globally unique per account) or `wa:sha256:{blob\_hash}"` if WA
 exposes a content hash.

The guard in step 2 (all four fields required) is what prevents the partial-metadata collision the first draft of this ADR allowed. The pure
 `compute_dedup_key(provider, *, document_id, source_id, provider_message_id,
 size_bytes, duration_s, height, width)` function that implements this tree
 gets its own unit tests in the implementation PR, including cases for
 partial/null metadata.

A **partial unique index** on `(account\_id, dedup\_key)` WHERE dedup_key IS NOT NULL
 replaces the current Tier-1 index and enforces per-account dedup at the DB
 level, regardless of provider.

Why recommended:

- Zero new bandwidth ? all keys are computed from metadata already extracted (honors ADR 0003's no-download principle).
- The provider prefix (`tg:` / `wa:`) makes cross-provider collisions structurally impossible.
- Existing `videos` rows migrate cleanly: `dedup\_key` is computed from their existing `document\_id` / metadata columns during the migration.
- Supersedes ADR 0006 cleanly: the three tiers become *one* column with provider-specific derivation rules, documented in the migration.

Costs:

- The migration must compute `dedup\_key` for every existing `videos` row (cheap ? pure SQL string building, no I/O).
- The `dedup\_key` derivation logic lives in code (a pure function per provider), not in the schema. This is a feature, but it means the derivation rules need their own unit tests.

Option 1B: content hash (SHA-256 of downloaded bytes)

`dedup\_key = sha256(downloaded\_content)`. Cryptographically strong,
 provider-agnostic, catches re-encodes that 1A misses.

Rejected ? violates ADR 0003 (metadata-first, no download in the hot path). Downloading every incoming video to hash it would multiply bandwidth and latency by orders of magnitude. Could be added later as an optional deep-inspect stage (deferred to Track F per #31).

Option 1C: provider-assigned only (drop Tier 2)

`dedup\_key = "tg:doc:{document\_id}"` always; no metadata fingerprint fallback. Simpler, but loses Tier-2 dedup for re-uploads that get a new `document.id` ? a real and common case (someone re-encodes and re-uploads the same video to a different channel).

Recommendation: 1A. It preserves the dedup quality of ADR 0006, generalizes cleanly, and honors the no-download constraint.

Question 2: `Policy` ? existing `filter\_config` ? rename, or wrap?

The codebase already has `filter\_config.config\_json` (one row per account, holding a `FilterConfig` JSON blob ? see `models.py` and ADR 0005). The umbrella introduces a `Policy` concept. They overlap. Two options:

Option 2A (recommended): `Policy` *generalizes* `filter\_config` (rename + extend)

`policies` is a new table; `filter\_config` is migrated into it. One account can own multiple policies (enables per-route filter overrides from ADR 0009), with a `DEFAULT` policy per account for the no-rules-configured case.

...

```
policies (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  account_id  INTEGER NOT NULL REFERENCES accounts(id),  
  name        TEXT NOT NULL,                -- e.g. "default", "reggae-route"  
  config_json TEXT NOT NULL,                -- the FilterConfig blob (Phase 1)  
  is_default  INTEGER NOT NULL DEFAULT 0,    -- one per account (UNIQUE partial idx)  
  created_at  TEXT NOT NULL,  
  updated_at  TEXT NOT NULL,  
  UNIQUE (account_id, name)  
);  
...
```

****Phase 1**** (this track): `config\_json` holds exactly today's `FilterConfig` JSON ? the rename is purely structural. ****Phase 2**** (Track F, deferred): `config\_json` grows to include topic preferences and security rules, but that's behind the same column so no further migration.

Migration: for each `filter\_config` row, insert a `policies` row with `name='default'`, `is\_default=1`, `config\_json=<the existing blob>`. Drop `filter\_config` after the migration (keep `filter\_config\_deprecated` for one release, mirroring ADR 0010's safety-net pattern).

****Why recommended:****

- `Policy` becomes the single home for "what to filter/enrich" ? no second overlapping concept. Today's `FilterConfig` is just the Phase-1 shape.
- Enables ADR 0009's per-route filter overrides (`routing\_rules.policy\_id`) without a second migration later.
- The existing `db.load\_filter\_config(account\_id)` / `save\_filter\_config` become thin shims over `policies WHERE account\_id=? AND is\_default=1`, mirroring the Track-A `User` shim pattern ? backwards-compatible.

****Costs:****

- One more table migration. `filter\_config` ? `policies` is a rename+extend.
- The `FilterConfig` Pydantic model stays (it's the Phase-1 config shape); `Policy` is the **container** (name, owner, default flag) around it. Two models, not one ? but they compose cleanly.

Option 2B: keep `filter\_config` as-is; `Policy` is a wrapping layer

Don't touch `filter\_config`. `policies` references a `filter\_config` row plus adds enrichment/security stages as separate columns.

****Rejected**** ? creates two overlapping concepts (`filter\_config` for the v1 rules, `policies` for "everything else"), which is exactly the conflation the umbrella exists to dissolve. We'd be back here in a year merging them.

****Recommendation: 2A.**** One concept, one table, phased growth via the same JSON column.

Proposed schema (assuming 1A + 2A)

`sources` (generalizes `channels`)

A configured place we ingest from. Maps 1:1 to today's per-account `channels` rows for Telegram; future providers add their own source kinds.

...

```
sources (
    id                INTEGER PRIMARY KEY AUTOINCREMENT,
    account_id        INTEGER NOT NULL REFERENCES accounts(id),
    provider           TEXT NOT NULL DEFAULT 'telegram',
    provider_source_id TEXT NOT NULL,          -- Telegram channel_id; future: WA group id
    title              TEXT,
```

6. user (2026-07-06T19:18:30.672Z)

```
0001-userbot-over-botapi.md
0002-sqlite-over-postgres.md
0003-metadata-first-probing.md
0004-pluggable-delivery-modes.md
0005-per-user-delivery-target.md
0006-content-dedup.md
0007-inline-config-commands.md
0008-bot-api-onboarding-front-door.md
0009-selective-channel-scanning.md
0010-identity-and-session-separation.md
0011-provider-neutral-domain-schema.md
---
main
```

7. assistant (2026-07-06T19:18:34.218Z)

ADR 0011 confirms the finding's evidence. Now let me verify the code claims: the pre-pipeline dedup in telethon_client.py and whether PolicyContext only sees a single policy.

8. user (2026-07-06T19:18:36.673Z)

```
7:      -> dedup via db.get_video()
8:      -> FilterEngine.evaluate(meta)
37:from tg_video_filter.filters import FilterEngine, VideoMeta
150:  engine_holder: dict[str, FilterEngine],
159:      2. dedups via two-tier content dedup (document.id, then metadata fingerprint),
160:      plus the existing (channel, message) instance dedup,
184:      # --- watched gate (ADR 0009, Track E) -----
185:      # Opt-in: a source is ingested only when watched=True. An absent
187:      # unwatched and lazily create a watched=0 row for /channels
189:      if not await db.should_ingest_source(conn, user.id, channel_id):
190:          return # skip unwatched / unknown source
192:      # --- three-tier dedup (cheapest first) ---
204:          "video_dedup_document_id user=%s doc_id=%s channel=%s msg=%s "
232:          "video_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s "
257:      result = engine_holder[ENGINE_KEY].evaluate(meta)
273:          # Lost a dedup race: a concurrent delivery of the same file
279:          "video_dedup_race user=%s channel=%s msg=%s doc_id=%s",
327:      This is the most reliable content-based dedup key: Telegram reuses the
342:      collapse them to a single int for dedup/storage. The exact subclass isn't
```

9. assistant (2026-07-06T19:18:45.318Z)

Confirmed: dedup runs at lines 192-256, before `evaluate` at line 257. The finding's evidence holds precisely. The ADR itself acknowledges dedup is "already handled by `content\_items.dedup\_key`" and that the pipeline runs "after dedup."

Let me verify the PolicyContext single-policy claim ? the ADR's dataclass shows `policy: Policy` (singular), and check the ADR's own text on the dedup stage scope.

The evidence is fully confirmed from the ADR text itself:

- Context section: "The pipeline runs **after** dedup and **before** delivery."
- Tier 3 item 3: "Duplicate suppression: enhanced dedup ... (already handled by `content\_items.dedup\_key`; this stage adds cross-policy duplicate detection within an account's delivery streams)."

- `PolicyContext` dataclass carries `policy: Policy` (a single owning policy) ? no cross-policy visibility.
- ADR 0011 Option 1A: partial unique index on `(account\_id, dedup\_key)` enforces per-account dedup at the DB level.

Now let me assess adversarially ? can the finding be refuted? The ADR does explicitly say "already handled by `content\_items.dedup\_key`; this stage adds cross-policy duplicate detection within an account's delivery streams." So the ADR *does* attempt to define a distinct scope ("cross-policy ... within an account's delivery streams"). The question is whether that scope is coherent/specified.

10. user (2026-07-06T19:18:47.226Z)

```
6:``destinations`` / ``routing_rules`` / ``delivery_attempts`` schema (ADR 0011).
38:CREATE TABLE IF NOT EXISTS accounts (
46:CREATE TABLE IF NOT EXISTS telegram_sessions (
55:CREATE TABLE IF NOT EXISTS channels (
64:CREATE TABLE IF NOT EXISTS videos (
81:CREATE TABLE IF NOT EXISTS filter_config (
88:-- the shims below); ``destinations`` / ``routing_rules`` / ``delivery_attempts``
92:CREATE TABLE IF NOT EXISTS sources (
105:CREATE TABLE IF NOT EXISTS content_items (
121:    dedup_key          TEXT NOT NULL,
125:    UNIQUE (account_id, dedup_key)
128:CREATE TABLE IF NOT EXISTS policies (
143:CREATE TABLE IF NOT EXISTS destinations (
155:CREATE TABLE IF NOT EXISTS routing_rules (
160:    destination_id INTEGER NOT NULL REFERENCES destinations(id),
164:    UNIQUE (account_id, source_id, destination_id)
167:CREATE TABLE IF NOT EXISTS delivery_attempts (
170:    destination_id INTEGER NOT NULL REFERENCES destinations(id),
178:    UNIQUE (content_item_id, destination_id)
255:# Pure function: computes the ``content_items.dedup_key`` value from metadata
259:# See ``compute_dedup_key_test`` in ``tests/test_db.py`` for the full matrix.
262:def compute_dedup_key(
273:    """Compute the provider-neutral ``dedup_key`` for a ``ContentItem``.
290:    ``dedup_key`` is worse than a loud failure at ingestion time.
303:    "Telegram dedup_key fallback requires source_id and provider_message_id "
310:    f"dedup_key for provider={provider!r} requires source_id and
provider_message_id"
382:    CREATE TABLE accounts (
403:    CREATE TABLE telegram_sessions (
437:    CREATE TABLE channels (
451:    CREATE TABLE videos (
474:    CREATE TABLE filter_config (
508:    Creates ``sources``, ``content_items``, ``policies``, ``destinations``,
509:    ``routing_rules``, ``delivery_attempts`` (all ``IF NOT EXISTS`` ? on fresh
562:    dedup_key, matched, seen_at)
585:    UPDATE content_items SET dedup_key = 'tg:doc:' || document_id
587:    AND dedup_key LIKE 'tg:msg:%'
590:    WHERE document_id IS NOT NULL AND dedup_key LIKE 'tg:msg:%'
599:    UPDATE content_items SET dedup_key =
605:    AND dedup_key LIKE 'tg:msg:%'
611:    AND dedup_key LIKE 'tg:msg:%'
```

11. assistant (2026-07-06T19:18:52.781Z)

This is decisive. Key facts:

1. `content\_items` has NO `content\_fingerprint` column ? the ADR's Tier-3 stage says it "compares by `content\_fingerprint`," but no such field exists in the schema (dedup_key at line 121 is the only dedup field). The ADR references a field that doesn't exist.
2. `delivery\_attempts` (line 167) is keyed by `(content\_item\_id, destination\_id)` with `UNIQUE (content\_item\_id, destination\_id)` at line 178 ? destination-scoped, not policy-scoped. There is

no "delivery stream per policy" concept in the schema; delivery is per-destination.

3. `PolicyContext` in the ADR carries a single `policy: Policy`, so a stage cannot see "other policies' delivery streams."

Let me confirm there's no content_fingerprint anywhere and check the ContentItem model.

12. user (2026-07-06T19:18:55.403Z)

```
src/tg_video_filter/backfill.py:192:             existing = await
db.get_video_by_content_fingerprint(
src/tg_video_filter/db.py:159:     policy_id     INTEGER REFERENCES policies(id),
src/tg_video_filter/db.py:718:             (account_id, source_id, policy_id, destination_id,
enabled, priority, created_at)
src/tg_video_filter/db.py:910:             policy_id=row["policy_id"],
src/tg_video_filter/db.py:1479:async def get_video_by_content_fingerprint(
src/tg_video_filter/db.py:1806:     "get_video_by_content_fingerprint",
src/tg_video_filter/models.py:249:     ``policy_id`` is NULL, the account's default policy is
used. When multiple
src/tg_video_filter/models.py:258:     policy_id: int | None = Field(
src/tg_video_filter/models.py:419:     def content_fingerprint(self) -> tuple[int, int, int, int]
| None:
src/tg_video_filter/telethon_client.py:222:             existing = await
db.get_video_by_content_fingerprint(
=== ContentItem fields ===
149:class ContentItem(BaseModel):
150-     """A normalized content record discovered from any :class:`Source` (ADR 0011).
151-
152-     Generalizes the legacy :class:`VideoRecord`. Phase 1 stores only videos
153-     (``content_type='video'``); ``text`` / ``link_url`` are landed now but stay
154-     NULL until later tracks populate them (avoids a future ``ALTER TABLE``).
155-     ``dedup_key`` is the provider-neutral dedup key (Option 1A: tiered lookup
156-     wrapping ADR 0006, computed by :func:`tg_video_filter.db.compute_dedup_key`).
157-     """
158-
159-     model_config = ConfigDict(frozen=True)
160-
161-     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
162-     account_id: int = Field(description="FK ? accounts.id")
163-     source_id: int = Field(description="FK ? sources.id")
164-     provider: str = Field(default="telegram", description="Provider namespace")
165-     provider_message_id: str = Field(
166-         description="Provider-native message id (Telegram message_id, ?)"
167-     )
168-     content_type: str = Field(default="video", description="'video' | 'link' | 'article'")
169-     text: str | None = Field(default=None, description="Caption / post text (nullable in
Phase 1)")
170-     link_url: str | None = Field(default=None, description="For link/article posts (Phase 1:
NULL)")
171-     media_ref: str | None = Field(default=None, description="Provider media reference")
172-     document_id: str | None = Field(
173-         default=None,
174-         description="Telegram document.id (kept for TG-specific Tier-1 lookups)",
175-     )
176-     duration_s: int | None = None
177-     width: int | None = None
178-     height: int | None = None
179-     size_bytes: int | None = None
```

13. assistant (2026-07-06T19:19:09.656Z)

Now I have full clarity. Findings:

1. `content\_fingerprint` exists as a **computed property on `VideoRecord`/`ContentItem`** (models.py:419, returns the 4-tuple `(size, duration, height, width)`) ? the legacy Tier-2

fingerprint. So the ADR's claim it "compares by `content\_fingerprint` across accounts" is describing exactly the pre-pipeline Tier-2 dedup that already runs (telethon_client.py:222 `get\_video\_by\_content\_fingerprint`), which the ADR itself concedes is "already handled by `content\_items.dedup\_key`."

2. `routing\_rules` has `policy\_id` (db.py:159) ? a policy is attached per route, and `delivery\_attempts` is keyed per `(content\_item\_id, destination\_id)`. So "cross-policy duplicate detection within an account's delivery streams" would mean: detecting the same content_item routed via different policies to different destinations. But `PolicyContext` (ADR) carries a single `policy`, giving a stage no visibility into other policies or into `delivery\_attempts`.

The finding holds. Let me assess adversarial refutation attempts:

****Refutation attempt 1:**** "The ADR distinguishes the scope ? it's not redundant, it's 'cross-policy within delivery streams.'" ? But that scope is undefined: (a) the data source isn't specified (delivery_attempts? content_items?), (b) PolicyContext structurally can't see cross-policy state, (c) it references `content\_fingerprint` which is the exact same metadata the pre-pipeline dedup_key already uses. So the ADR names a distinct scope but provides no mechanism ? the gap is real.

****Refutation attempt 2:**** "Design docs don't need mechanism detail." ? But this ADR is explicit that it "defines the design contract the phases code against," and the `PolicyContext` dataclass IS the contract. A stage that needs cross-policy/cross-destination state is uncodeable against the given contract. That's a genuine internal inconsistency, not merely deferred detail.

The finding is CONFIRMED. On severity: it's an unspecified interaction / internal inconsistency in a draft, not a false claim causing data loss or breaking an F-phase outright (F4 is URL reputation; dedup suppression isn't even assigned a phase in the F1-F7 table, which itself reinforces the finding ? Tier-3 dedup appears in the tier diagram and stage list but has NO corresponding implementation phase). Medium is correct.

14. user (2026-07-06T19:19:22.481Z)

Structured output provided successfully